

whitenoise -- Exoplanet Guide

SWNA on TESS Transit Light Curves | wn.exoplanet submodule

Framework: Bernido & Carpio-Bernido (2015) - World Scientific

SWNA on TESS Transit Light Curves * wn.exoplanet submodule

Framework: Bernido & Carpio-Bernido (2015) * World Scientific

Overview

This guide covers applying Stochastic White Noise Analysis (SWNA) to exoplanet transit photometry from the TESS mission. The stochastic variable is the normalized flux within a window centred on a transit, sampled at the TESS SPOC 2-minute cadence. The memory parameter μ extracted by SWNA characterizes the noise regime of the transit-window flux -- the same physical quantity computed for RadioJOVE bursts and genomic distance sequences, applied here to photometric time series.

The full pipeline has two stages:

Stage 1 -- Preprocessing (light curve -> clean transit-window CSV):

```
download_transit_lc()
->
find_empirical_midpoint()
->
extract_transit_window()
->
CSV file
```

Stage 2 -- SWNA analysis (automated, using this package):

```
analyze_transit()
plot + CSV
->
batch_analyze_transits()
summary DataFrame
```

Use `analyze_transit()` for a single transit or `batch_analyze_transits()` to process an entire pipeline/Planet/ folder tree at once. All functions are accessible via `wn.exoplanet.*`.

1 * Setup

```
import whitenoise as wn

# Verify the submodule is available
print(wn.__version__)
print(dir(wn.exoplanet))
```

Path note: If whitenoise is installed via `pip install -e .` from

the package root, no path configuration is needed. If running from a project that embeds the package, add to the top of your script:

```
import sys, os
sys.path.insert(0, 'path/to/whitenoise')
```

Optional dependency: Downloading light curves requires `lightkurve`, which is not a core `whitenoise` dependency. Install it with:

```
pip install whitenoise-swna[exoplanet]
```

Everything downstream of a raw (time, flux) array -- window extraction, CSV I/O, SWNA analysis -- has no `lightkurve` dependency at all.

2 * Stage 1 -- From Light Curve to Transit-Window CSV

Step 1 -- Download and clean a light curve

```
wn.exoplanet.download_transit_lc(target, cadence=120, period_days=None, ...) -> (time, flux,
```

```
time, flux, meta = wn.exoplanet.download_transit_lc('WASP-078', period_days=2.175)
print(f"Sector {meta['sector']}: {len(time)} points, ~{meta['n_transits']:.1f} transits")
```

This downloads every available TESS SPOC sector for target, cleans each with `clean_lc()` (NaN removal, sigma-clipping, median normalization), and returns the sector with the most estimated transits (longest baseline / `period_days`).

If you already have a `lightkurve.LightCurve` object (e.g. from your own search/download call), clean it directly:

```
wn.exoplanet.clean_lc(lc, sigma=3.0) -> (time, flux)
```

```
import lightkurve as lk
sr = lk.search_lightcurve('WASP-078', mission='TESS', cadence=120, author='SPOC')
lc_collection = sr.download_all(quality_bitmask='default')
time, flux = wn.exoplanet.clean_lc(lc_collection[0])
```

Step 2 -- Estimate the transit duration (T14)

```
wn.exoplanet.estimate_T14(P_days, Rs, Rp, a_AU, inc_deg=90.0) -> float
```

```
T14_d = wn.exoplanet.estimate_T14(
    P_days=2.175,    # orbital period (days)
    Rs=2.350,        # stellar radius (solar radii)
    Rp=2.060,        # planet radius (Jupiter radii)
    a_AU=0.03670,    # semi-major axis (AU)
)
print(f'T14 = {T14_d*24:.2f} hours')
```

With the default `inc_deg=90.0` (central transit, impact parameter $b=0$), this reduces to the standard $T14 = (P/?) * \arcsin((R_s + R_p)/a)$ formula. Pass the catalog inclination for a non-central-transit estimate.

Step 3 -- Find the empirical transit midpoint

```
wn.exoplanet.find_empirical_midpoint(t, f, t_lo, t_hi, T14_d) -> (t_mid, depth, snr, method)
```

```
t_mid, depth, snr, method = wn.exoplanet.find_empirical_midpoint(
    time, flux, t_lo=1234.5, t_hi=1234.7, T14_d=T14_d,
)
print(f't_mid={t_mid:.6f} depth={depth:.5f} SNR={snr:.1f} [{method}]')
```

Locates a transit midpoint within `[t_lo, t_hi]` without a forward model, using a three-step hybrid: a sliding-window scan (500 candidates, robust down to SNR approx 1), parabolic refinement for sub-cadence precision, and a T14-informed flux-weighted centroid. The three estimates are blended by SNR -- see the function docstring for the exact blend rule.

No forward model needed: Unlike a batman/chi-squared transit fit, this approach makes no assumption about limb darkening or transit shape -- it only needs a rough search window and the expected duration T14. It is intentionally simple and robust rather than maximally precise.

Step 4 -- Extract a gap-filled transit window

```
wn.exoplanet.extract_transit_window(time, flux, t_mid, T14_d, window_periods=1.5, ...) -> di
```

```
window = wn.exoplanet.extract_transit_window(time, flux, t_mid, T14_d)
if window is None:
    print('rejected: out of bounds, too few points, or low coverage')
else:
    print(f'{len(window["time"])} points, {window["n_filled"]} NN-filled, "
        f"coverage={window["coverage"]:.2f}')
```

Cuts out the window `t_mid +/- window_periods*T14_d` (the default `window_periods=1.5` gives the "3T" window used throughout this guide), fills small gaps ($\leq \text{max_fill_gap_min}$, default 60 min) with nearest-neighbour flux, and returns None if the window falls outside the light curve, has fewer than `min_points` (default 5), or covers less than `min_coverage` (default 50%) of the expected uniform-cadence grid.

Return key	Type	Description
t_lo, t_hi	float	Window bounds (days)
time, flux	np.ndarray	Gap-filled arrays within the window
is_filled	np.ndarray[bool]	True for NN-filled points
gap_info	list	(t_start, t_end, gap_minutes) per real g
coverage	float	Fraction of expected uniform-cadence poi
n_filled	int	Number of NN-filled points

Step 5 -- Write the whitenoise-format CSV

```
wn.exoplanet.write_pipeline_csv(time, flux, t_mid, out_path) -> str
```

```
wn.exoplanet.write_pipeline_csv(
    window['time'], window['flux'], t_mid,
    'pipeline/WASP-078/WASP-078_T01.csv',
)
```

Output format (2 columns, no comments -- this is exactly what wn.analyze() reads):

```
time_min_from_mid, flux
-45.200000, 0.99980000
-43.200000, 0.99975000
...
```

No metadata comments in this file. wn.read_csv() does not support #-commented header lines, so write_pipeline_csv() writes a strictly 2-column file. Track provenance (planet, sector, midpoint-finding method, SNR) separately -- e.g. as one row per transit in your own DataFrame/CSV -- rather than embedding it in the pipeline CSV.

Reading it back / listing a folder

```
wn.exoplanet.read_pipeline_csv(path) -> (time_min_from_mid, flux)
```

```
wn.exoplanet.list_transit_csvs(folder, pattern='*.csv') -> list[str]
```

```
t_min, flux = wn.exoplanet.read_pipeline_csv('pipeline/WASP-078/WASP-078_T01.csv')
paths = wn.exoplanet.list_transit_csvs('pipeline/WASP-078/')

```

3 * Single Transit Analysis -- Step by Step

Use this approach when you want to inspect each stage individually.

Step 1 -- Read the CSV

```
t_min, flux = wn.exoplanet.read_pipeline_csv('pipeline/WASP-078/WASP-078_T01.csv')
```

Step 2 -- Compute MSD

```
lags, msd = wn.compute_msd(flux)
print(f"MSD: {len(lags)} lags, MSD[1]={msd[0]:.6f}, MSD[-1]={msd[-1]:.6f}")
```

No detrending: Transit-window studies in this package typically use `detrend_method=None` -- the window is short enough (a few hours) that instrumental trends are negligible relative to the transit signal itself.

Step 3 -- Fit a model

```
fit = wn.fit_msd(lags, msd, model='exponential')
if fit:
    print(fit.summary())
```

Step 4 -- Plot

```
fig = wn.plot_msd(result) # or wn.plot_diagnostics(result) for the full 2x2 panel
fig.savefig('transit_msd.png', dpi=150, bbox_inches='tight')
```

4 * Full Single-Transit Pipeline (One Call)

```
wn.exoplanet.analyze_transit(time, flux, t_mid, T14_d, model='exponential',
                             detrend_method=None, csv_path=None, output_dir=None,
                             dataset_name='transit', verbose=True) -> dict
```

Runs the full chain -- window extraction, optional CSV export, `wn.analyze()`, and a diagnostic plot -- in a single call.

```

out = wn.exoplanet.analyze_transit(
    time, flux, t_mid, T14_d,
    model='exponential',
    csv_path='pipeline/WASP-078/WASP-078_T01.csv',
    output_dir='results/swna/',
    dataset_name='WASP-078_T01',
)

result = out['result']
if result and result.fit:
    print(f"mu = {result.fit.params['mu']:.4f}")
    print(f"R2 = {result.fit.r_squared:.4f}")

out['figure']      # diagnostic plot (wn.plot_diagnostics output)
out['window']      # dict from extract_transit_window(), or None if rejected
out['row']         # summary dict for building a DataFrame

```

Return key	Type	Description
window	dict None	Output of extract_transit_window()
result	AnalysisResult None	Full SWNA result (None if extraction/fit
figure	Figure None	Diagnostic plot (wn.plot_diagnostics out
row	dict None	Summary row for DataFrame construction

csv_path is optional. If omitted, the extracted window is analyzed via a temporary file that is deleted afterward -- useful for quick exploration without writing permanent pipeline CSVs.

5 * Batch Analysis

```

wn.exoplanet.batch_analyze_transits(pipeline_dir, planets=None,
                                    models=('cosine', 'exponential'), detrend_method=None,
                                    output_dir=None, verbose=True) -> pd.DataFrame

```

Walks a pipeline_dir/<Planet>/<Planet>_T##.csv tree -- one subfolder per planet, each containing 2-column transit CSVs written by write_pipeline_csv() -- running every requested model on every transit and saving per-transit diagnostic plots plus a summary CSV/Excel.

```

df = wn.exoplanet.batch_analyze_transits(
    'data/selected_transits/pipeline/',
    models=('cosine', 'exponential'),
    output_dir='results/swna/',
)

print(df[['planet', 'transit', 'model', 'mu', 'r_squared', 'regime']])

```

Restrict to specific planets:

```
df = wn.exoplanet.batch_analyze_transits(
    'data/selected_transits/pipeline/',
    planets=['WASP-078', 'WASP-072'],
    models=('exponential',),
)
```

DataFrame column	Description
planet	Subfolder name
transit	CSV filename stem
model	SWNA model used
r_squared	Goodness of fit R2
regime	Model-family classification (see §7)
mu, beta/nu/H, N	Fitted parameters (vary by model)

6 * Complete Workflow Example

```

import whitenoise as wn

# ?? Configuration ?????????????????????????????????????????????????????????????
TARGET    = 'WASP-078'
P_DAYS    = 2.175
T14_d     = wn.exoplanet.estimate_T14(P_days=P_DAYS, Rs=2.350, Rp=2.060, a_AU=0.03670)
OUT_DIR   = 'results/swna/'

# ?? Step 1: download and clean ?????????????????????????????????????????????
time, flux, meta = wn.exoplanet.download_transit_lc(TARGET, period_days=P_DAYS)
print(f"Sector {meta['sector']}, {len(time)} points")

# ?? Step 2: find each expected transit and extract a window ?????????????????
n = 0
for t_mid_guess in expected_midpoints:    # from your own ephemeris
    t_lo, t_hi = t_mid_guess - P_DAYS/4, t_mid_guess + P_DAYS/4
    t_mid, depth, snr, method = wn.exoplanet.find_empirical_midpoint(time, flux, t_lo, t_hi, T14_d)
    if t_mid is None:
        continue
    n += 1
    wn.exoplanet.analyze_transit(
        time, flux, t_mid, T14_d,
        model='exponential',
        csv_path=f'data/pipeline/{TARGET}/{TARGET}_T{n:02d}.csv',
        output_dir=OUT_DIR,
        dataset_name=f'{TARGET}_T{n:02d}',
    )

# ?? Step 3: batch summary across every planet folder ?????????????????????
df = wn.exoplanet.batch_analyze_transits(
    'data/pipeline/',
    models=('cosine', 'exponential'),
    output_dir=OUT_DIR,
)
print(df[['planet', 'transit', 'model', 'mu', 'r_squared', 'regime']])

```

7 * Choosing a Model

The cosine model fits poorly on transit MSD curves. Transit-window flux MSD saturates at high lags -- the cosine model's oscillatory form cannot follow that saturation, producing very negative R2 on the pure-MSD fit (the N*MSD-scaled R2 is usually still reported, but treat cosine results for transits as a secondary comparison, not the primary result). Use model='exponential' for the primary analysis.

Model	Role	Use for transits?
exponential	Monotonically saturating MSD, mu-based	Recommended (primary)
cosine	Oscillating MSD, alpha-based diffusive c	Secondary comparison only

Interpreting mu (exponential model)

Classification comes from the power-law scaling exponent ($\mu-1$)/2:

$\mu = 1$ (exponent = 0) is memoryless; $\mu > 1$ (exponent > 0) is non-Markovian long memory; $\mu < 1$ (exponent < 0) is non-Markovian short memory. Source: Tey et al. (2024, Physica Scripta 100, 015243).

8 * When the Fit is Poor

Symptom: window is rejected (extract_transit_window returns None)

Check, in order: is $t_{\text{mid}} \pm \text{window_periods} \times T_{14_d}$ within the light curve's time range? Are there fewer than `min_points` observed points in the window (short baseline, or too small a window_periods)? Is coverage below `min_coverage` after gap-filling (a real gap wider than `max_fill_gap_min` sits inside the window)? Widen `max_fill_gap_min` cautiously -- gaps that are too wide to fill meaningfully should stay excluded.

Symptom: find_empirical_midpoint returns None (method 'TOO_FEW' or 'SCAN_FAILED')

The search window $[t_{\text{lo}}, t_{\text{hi}}]$ has fewer than 5 observed points, or every scan candidate has fewer than 3 points in its $\pm T_{14}/2$ sub-window. Widen the search window or confirm the light curve actually covers the expected transit time.

Symptom: R2 is very negative on the pure-MSD fit

This is expected for the cosine model on transit data (see §7) -- check the $N \times \text{MSD}$ -scaled R^2 instead, or switch to `model='exponential'`.

Symptom: good shape but low R2

The optimizer found a local minimum. Inspect the MSD plot and supply custom initial parameters via `wn.fit_msd(lags, msd, model='exponential', p0=[mu_guess, beta_guess])`.

9 * Function Reference

Function	Location	Description
<code>estimate_T14(P_days, Rs, Rp, a_AU, inc_d)</code>	<code>wn.exoplanet</code>	Total transit duration (days)
<code>window_has_gap(time, t_lo, t_hi, max_gap)</code>	<code>wn.exoplanet</code>	Check for a gap wider than threshold in
<code>fill_gaps_nn(time, flux, t_lo, t_hi, ...)</code>	<code>wn.exoplanet</code>	Nearest-neighbour gap-fill on a uniform-
<code>find_empirical_midpoint(t, f, t_lo, t_hi)</code>	<code>wn.exoplanet</code>	Locate a transit midpoint without a forw
<code>extract_transit_window(time, flux, t_mid)</code>	<code>wn.exoplanet</code>	Windowed, gap-filled, quality-gated tran
<code>clean_lc(lc, sigma=3.0)</code>	<code>wn.exoplanet</code>	lightkurve LightCurve -> clean (time, fl
<code>download_transit_lc(target, cadence=120,</code>	<code>wn.exoplanet</code>	Download + clean the best TESS SPOC sect
<code>write_pipeline_csv(time, flux, t_mid, ou</code>	<code>wn.exoplanet</code>	Write the 2-column pipeline CSV format
<code>read_pipeline_csv(path)</code>	<code>wn.exoplanet</code>	Read a pipeline CSV back into arrays
<code>list_transit_csvs(folder)</code>	<code>wn.exoplanet</code>	List all pipeline CSVs in a folder, sort

<code>analyze_transit(time, flux, t_mid, T14_d</code>	<code>wn.exoplanet</code>	Full SWNA pipeline for one transit
<code>batch_analyze_transits(pipeline_dir, ...</code>	<code>wn.exoplanet</code>	Batch analysis over a planet-folder tree
<code>compute_msd(x)</code>	<code>wn</code>	Empirical MSD (lags 1 to N/2)
<code>fit_msd(lags, msd, model)</code>	<code>wn</code>	Fit any SWNA model; returns <code>FitResult</code>
<code>list_models()</code>	<code>wn</code>	Print all 16 SWNA models and status
<code>plot_diagnostics(result)</code>	<code>wn</code>	2x2 diagnostic figure: time series, MSD,

whitenoise * Exoplanet SWNA Guide *

Framework: Bernido C C, Carpio-Bernido M V (2015). Methods and Applications of White Noise Analysis in Interdisciplinary Sciences. World Scientific.